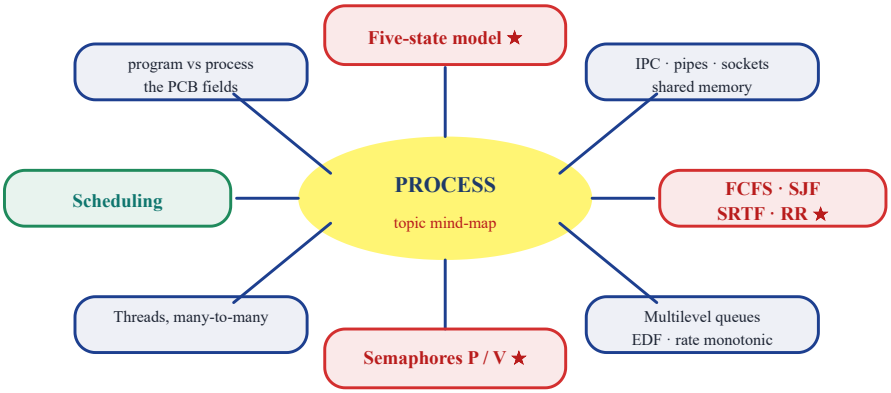
Process & Scheduling

PCB · five states · IPC & semaphores · CPU scheduling: exam-ready in 13 sheets



↑ the whole topic on one page: red = high-yield, green = the idea the scheduling criteria hang from

Syllabus coverage

The syllabus line is "**Process: process model and implementation, Inter-Process Communication (IPC), classical IPC problems, process scheduling, multiprocessing and time-sharing**". Deadlock, memory management and file systems have their own lessons.

Neighbouring lessons. Readers–writers, dining philosophers and the sleeping barber are synchronisation problems built on these primitives; deadlock prevention, Banker's algorithm and recovery are a separate lesson.

High-yield list

① program = file · process = in memory
② PCB: PID, state, PC, registers, sched, mem, accounts
③ new → ready → running → waiting → terminated
④ ready wants the CPU · waiting wants an event
⑤ long-, medium-, short-term scheduler
⑥ threads share the address space
⑦ IPC: shm, pipe, FIFO, socket, signal
⑧ semaphore P = −1, V = +1
⑨ TAT = C − arrival · WT = TAT − burst
⑩ SJF best average wait · RR = q turns ★

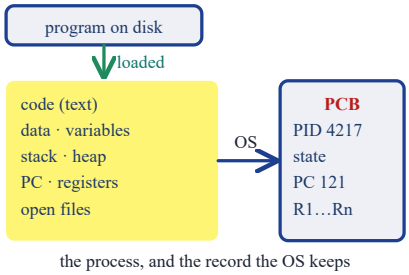
Legend: **yellow** = key term · **red circle** = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

Contents

✓ Program, process & the PCB … p. 2
✓ Five states & the queues … p. 3
✓ Creation & termination … p. 4
✓ Threads & their models … p. 5
✓ IPC mechanisms … p. 6
✓ Semaphores, producer–consumer … p. 7
✓ Criteria & preemption … p. 8
✓ FCFS, SJF, priority … p. 9
✓ SRTF & round robin … p. 10
✓ Multilevel queues, EDF … p. 11
✓ Night-before p. 12 · Practice p. 13

1 Program vs Process & the PCB

Definition: A **program** is a passive file of instructions on disk. A **process** is that program **loaded into memory and executing**, with its code, data, program counter, the CPU registers it owns and its open files. **A program is static, a process is dynamic** — one program may be many processes at once.



one program, one instance

Point	Program	Process
Nature	passive, static	active, dynamic
Memory	disk is enough	needs main memory
Identity	one file name	**PID**, many instances
Life	as long as the file	new → running → terminated

PCB field	Contents	Why the OS keeps it
Process state	New · Ready · Running · Waiting · Terminated	picks **the queue**
Program counter	address of the next instruction	resume **exactly**
CPU registers	accumulator, base, limit, stack, R0…Rn	unfinished arithmetic
CPU-scheduling info	priority, queue pointers	who runs next
Memory-management info	page-table base, segment-table pointer	address translation
Accounting info	user & kernel time, I/O counts, children	billing, quotas, cleanup

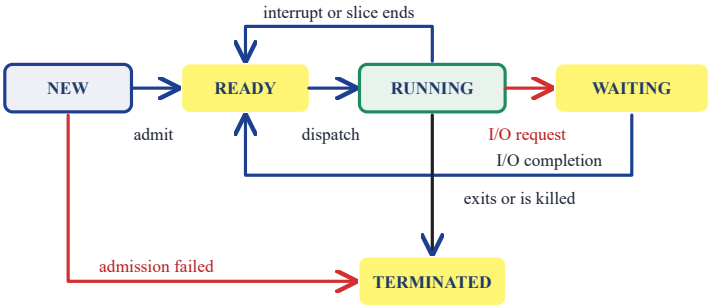
Memory trick: The PCB is the process's **passport**: the **PID** says who it is, the **state** says where it is, the **PC and registers** remember exactly where it stopped; the rest says what it owns.

Exam trap: (i) The PCB is a **kernel** structure made by the OS; the process never writes into it. (ii) A process is **not** its PCB — terminating frees both. (iii) A PID is unique among **running** processes and may be **reused** afterwards.

Worked example ✍ read a PCB dump: PID 4217, state **Waiting**, PC 121, base 240, limit 460, user time 128, kernel time 12. **(a)** It is **not** in the ready queue — it waits for an I/O event. **(b)** Legal addresses = **240 … 459**. **(c)** CPU used = 128 + 12 = **140** units. **(d)** The PID is not a scheduling key by itself; **priority** is.

2 The Five-State Model & the Queues

Definition: A process is in exactly one of **five** states: **new** (being created), **ready** (in memory, waiting for the CPU), **running** (holds the CPU), **waiting / blocked** (waiting for an event that is not the CPU) and **terminated** (finished, resources reclaimed).



seven transitions; the CPU scheduler only ever picks from **ready**

From → To	Trigger	By whom
New → Ready	admitted	long-term scheduler
Ready → Running	CPU allocated	short-term scheduler
Running → Ready	interrupt, slice ended, higher priority ready	short-term scheduler
Running → Waiting	I/O request or event wait	the process itself
Waiting → Ready	I/O done, event occurred	interrupt handler
Running → Terminated	finished, fatal error, killed	the process itself
New → Terminated	admission refused	long-term scheduler

Queue	Holds / drained by
Job queue	not yet admitted; long-term scheduler
Ready queue	in memory, waiting for the CPU; short-term scheduler
Device queues	waiting for that I/O; the device controller

Exam trap: Three schedulers exist: **long-term** (job queue), **medium-term** (swapping a process out and in) and **short-term** (the CPU). Only the CPU scheduler runs constantly.

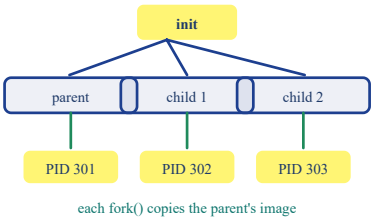
Memory trick: "**Ready** asks for the **CPU**, **waiting** asks for **something else**." Both return the same way — the event ends and the process goes back to the **ready** queue, never straight to the CPU.

Exam trap: Waiting **only** for the CPU means **ready**, not waiting — the single most asked state question. A process in the device queue is waiting even though it is in memory.

Worked example ✍ trace one process: You type a command. **(a)** new → ready when the long-term scheduler admits it. **(b)** ready → running when the CPU scheduler dispatches it. **(c)** running → waiting the moment it issues the print request. **(d)** waiting → ready when the printer finishes. **(e)** Earlier a clock interrupt ended its slice: that is running → ready, **not** running → waiting.

3 Process Creation & Termination

Definition: **Creation**: the OS takes a PID, allocates and fills a PCB, sets up an address space, attaches open files, and puts the PCB in the **ready queue**. **Termination**: the state is marked terminated, memory and files are released, and the exit status goes to the parent.



every child has exactly **one** parent

Step of creation	What happens
1. Get a PID	from the PID table; reused after release
2. Allocate the PCB	memory reserved for the control block
3. Initialise it	state, **PC at the entry point**, accounts = 0, queue pointers
4. Address space	copy the parent's image; build page/segment table
5. Open files	descriptors point at the same inodes
6. Resources	I/O, memory and CPU quota
7. Ready the context	stack and registers for the first dispatch
8. Enqueue	PCB → **ready queue**, never straight to running

Way a process ends	Typical cause
Normal exit	main() returns or `exit(0)`; the value becomes the exit status
Error exit	fatal error — divide by zero, invalid memory reference
Killed by another process	a signal such as SIGKILL, which cannot be caught
Killed by the OS	memory limit exceeded, protection fault, shutdown

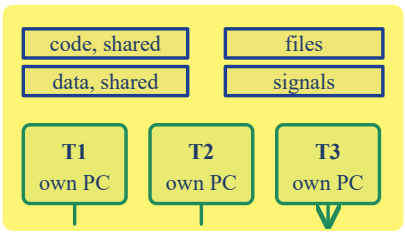
Exam trap: A child that finished before its parent collected it stays as a **zombie**: finished, but its exit status and table entries remain until the parent waits. A **child whose parent dies first** is an **orphan**, adopted by init.

Memory trick: **"Born ready, die finished."** Creation ends in the ready queue; termination frees the PID and the PCB. Because init adopts orphans, no zombie is left uncollected.

Worked example ✍️ a shell session: You type `ls > out.txt`. **(a)** The shell forks twice — once for `ls`, once for the right-hand side of the redirection. **(b)** The child opens the file before `exec`; the inode is **shared**, not copied. **(c)** `exec` replaces the image but **keeps the PID**. **(d)** When `ls` exits, the shell's `wait()` reaps it, so the zombie lives milliseconds.

4 Threads: Sharing One Address Space

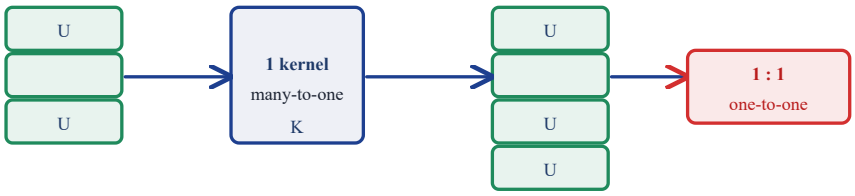
Definition: A **thread** is the unit of **execution** inside a process. Threads of one process **share the address space, code, data, open files and signals**; each keeps its own **stack, registers, program counter and thread-local storage**.



one process, three threads, one memory map

own stacks, one address space

Point	Process	Thread
Address space	its own, protected	**shared with siblings**
Private state	everything	stack, registers, PC, TLS
Creation cost	heavy: the image is copied	cheap: a thread control block
Communication	needs IPC	just read the shared variables
Failure effect	contained	a bad write can kill **all** threads
In the scheduler	always a separate entity	depends on the model below



many-to-many: $\overline{m}$ user threads on $\overline{n}$ kernel threads, $m \geq n$

the three multithreaded models — the OS always schedules **kernel** threads

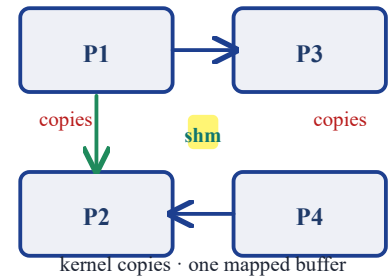
Level	Where it lives	What it gives
User-level	a library inside the process	cheap switch, but one blocking call blocks **the whole process**
Kernel-level	the OS keeps a real thread per entity	a blocking call blocks **one** thread; true parallelism on many cores

Exam trap: In **many-to-one** a blocking call stops **all** threads and there is **no parallelism**; in **one-to-one** parallelism is real but every thread costs a PCB and a kernel stack.
Thread scheduling ≠ process scheduling in the first two models: the kernel schedules kernel threads, the library schedules user threads.

Memory trick: "**Threads share, they do not copy.**" If you have to say `fork()` it is a **process**; if you can simply assign to a global it is a **thread**.

5 Inter-Process Communication

Definition: **IPC** lets one process **send data to** another and **coordinate** with it. Two families: **message passing** (copies move through the kernel) and **shared memory** (fastest — no copy — but it carries **no** synchronisation at all).



message passing versus shared memory

Mechanism	Kind	Notes
Shared memory	shared	fastest; **you add your own locking**
Anonymous pipe	message	byte stream, only **related** processes
Named pipe (FIFO)	message	filesystem name, **unrelated** processes
Message queue	message	kernel list of **discrete** messages
Socket	message	named endpoint, local **or** TCP/IP
Signal	message	one-way **notice**; SIGKILL uncatchable
Mailbox	message	indirect: a named drop box

Choose it when	Cost
Shared memory for huge buffers — video frames, indexes	every lock is yours to write
Pipe / FIFO for a stream of bytes, filtered in flight	buffer-limited, no message boundaries
Message queue for discrete jobs that keep their order	fixed maximum queue length
Socket to reach another machine or a service	serialisation, an address space per connection
Signal for "something changed — re-read your state"	no payload; handler runs at an odd moment

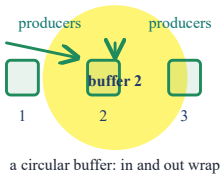
Exam trap: **Pipes serve related processes only** — a shell pipeline, a parent and its child. Two unrelated programs need a **named FIFO**, a socket or shared memory. Shared memory gives **no** mutual exclusion: two racing writers lose data.

Memory trick: "**Copy** means message passing, **map** means shared memory, **named** means unrelated." If the data never moves, a mutex's job is to make it safe.

Worked example ✍ pick the mechanism: **(a)** `sort < file | head` — anonymous **pipe**. **(b)** A logger daemon and 40 unrelated services — **named FIFO** or **socket**. **(c)** A video server writing 4 MB frames to two decoders — **shared memory** plus a mutex. **(d)** "Your config file changed" — a **signal**, then the process re-reads the file itself: a signal is a nudge, not a transfer.

6 Semaphores, Mutexes & Producer–Consumer

Definition: A **semaphore** (Dijkstra, 1965) is a non-negative integer plus a waiting queue, touched only by two **atomic** operations: **P (wait): if S = 0 block, then S ← S − 1** and **V (signal): S ← S + 1, wake one waiter**. A **binary** semaphore — a mutex — stays at 0 or 1.



a circular buffer: in and out wrap

two slots used, so empty = 0

Construct	Value	Owner?	Best for
Semaphore	integer ≥ 0	**no** — any process may V it	counting resources, ordering
Mutex	0 or 1	**yes** — only the holder unlocks	a critical section
Monitor	implicit, one thread inside	automatic	data + procedures + condition variables

The solution ✍ three semaphores: For a buffer of N slots: **mutex = 1** (one process inside the buffer code), **empty = N** (free slots), **full = 0** (filled slots).

```
producer:                    consumer:
wait(empty);   wait(full);
wait(mutex);    wait(mutex);
  put item;        take item
signal(mutex);  signal(mutex);
signal(full);    signal(empty);
```

The **order of the two outer waits is the whole problem**: a producer claims a free slot **before** the mutex, a consumer a filled slot before the mutex. Reverse them and both can sit on the mutex for ever.

Trace, buffer of 2	Operation	mutex	empty	full	buffer (in → out)
—	start	1	2	0	empty
1	P1 puts **x**	1	1	1	x
2	P2 puts **y**	1	0	2	x y
3	P3 wants to put **z**	1	**0**	2	**P3 blocks**
4	C1 takes **x**	1	1	1	y
5	C2 takes **y**	1	2	0	empty
6	P3 wakes, puts **z**	1	1	1	z

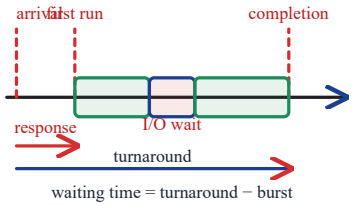
Memory trick: **"empty counts spaces, full counts items, mutex counts permission, and empty + full always = N."** In every row above empty + full = 2; if it is not, the trace is wrong.

Exam trap: **P decreases, V increases** — the semaphore pays out on P. A counting semaphore starting at 7 with 10 P and 6 V ends at **7 − 10 + 6 = 3**. Unlocking a mutex you do not own is meaningless.

7 Scheduling Criteria & Preemption

Definition: CPU scheduling chooses **which ready process runs next** and **for how long**. No algorithm wins on every measure, so scheduling is judged against **five criteria** — and the algorithm a system picks depends on which one that system cares about.

Criterion	Definition & formula	Wanted
CPU utilisation	busy fraction = **1 − idle fraction**	as **high** as possible
Throughput	processes completed **per unit time**	as **high** as possible
Turnaround time	**completion − arrival**, the whole life	**as low as possible**
Waiting time	**turnaround − CPU burst**	**as low as possible**
Response time	**first response − arrival**	low — what an **interactive** user feels



everything not spent on the CPU is waiting

Point	Non-preemptive	Preemptive
A running process	keeps the CPU till it finishes **or blocks**	can be **interrupted** and requeued
Needs	nothing special	a timer interrupt, OS support
Overhead	low — few switches	one switch per preemption
Response time	can be **poor**: a long job blocks all	good, bounded by q
Trap	never preempt **inside a critical section**	doing so corrupts the state

Multiprocessing vs time-sharing — the distinction the syllabus asks for:

Multiprogramming: several programs in memory, the CPU switches between them

Multiprocessing: several **processors**, genuine parallelism

Time-sharing: one CPU shared in **time slices** — the classic time-sharing OS is round robin

Multitasking: several tasks on one CPU, the idea behind time-sharing

Real-time: deadlines matter more than throughput

Batch: throughput maximised, response ignored

Exam trap: Waiting time is **not** turnaround time, and response time is **not** waiting time. Burst 10, arrival 2, completion 20: TAT = **18**, WT = **8**, response uses the **first** dispatch.

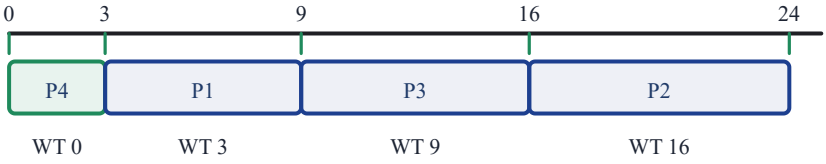
Memory trick: "**Turn**around = arri**b**al to fini**sh**. **W**aiting = turnaround minus **b**urst. **R**esponse = arrival to first **r**un."

Worked example ✍️ all three times: A job arrives at t = 2, is first dispatched at t = 5, blocks for I/O at t = 9, is dispatched again at t = 12 and completes at t = 24; its CPU burst totals 12. **(a)** Turnaround = 24 − 2 = **22**. **(b)** Waiting = 22 − 12 = **10**; the gaps confirm it — (5−2) + (12−9) = 6 while queued, plus 4 in its last dispatch. **(c)** Response = 5 − 2 = **3**. **(d)** Non-preemptively its response would be 0 — but its neighbours would pay.

8 Non-Preemptive: FCFS, SJF & Priority

Definition: A process keeps the CPU until it finishes or blocks. **FCFS** serves arrival order, **SJF** always picks the shortest CPU burst, **priority scheduling** picks the highest-priority process and breaks ties by FCFS.

Algorithm	Rule	Good	Bad
FCFS	arrival order	fair in CPU time, simple, no starvation	**convoy effect**: one long job delays every short one
SJF	shortest burst first	**minimum average waiting time**, provably optimal	needs bursts in advance; long jobs may **starve**
SRTF (preemptive)	shortest *remaining* time	also optimal for average waiting	frequent switches; needs burst estimates
Priority	highest priority first	expresses importance; quick response for urgent work	**starvation** for low-priority processes
Round robin (preemptive)	one **time quantum** each, in turn	fair CPU share, bounded response time	switch overhead if the quantum is small



SJF on bursts 6, 8, 7, 3 ms: P4 0–3, P1 3–9, P3 9–16, P2 16–24

Worked example 1 ✍️ FCFS: Bursts **24, 3, 3**, all at t = 0. P1 0–24, P2 24–27, P3 27–30. Waiting 0, 24, 27 → average 51/3 = **17 ms**. Two 3 ms jobs sat behind a 24 ms job — the convoy.

Worked example 2 ✍️ SJF: Bursts P1 = 6, P2 = 8, P3 = 7, P4 = 3. Order P4 0–3, P1 3–9, P3 9–16, P2 16–24. Waiting 0, 3, 9, 16 → average 28/4 = **7 ms**. FCFS gives 0, 6, 14, 21 → **10.25 ms**.

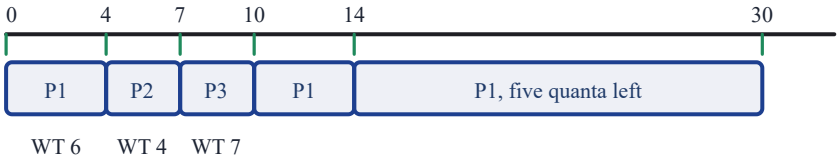
Exam trap: SJF is optimal for **average waiting time** — not for turnaround or response, and not for weighted combinations. It also needs the burst **known**; in practice the OS estimates it from the process's history, which is why SRTF is really "shortest remaining time *next*".

Memory trick: "**Short first, long last**" for SJF; the **convoy** is FCFS's disease; **starvation** is priority's, and its cure is **ageing** — the effective priority rises with waiting time.

Ageing, precisely: After each wait interval the OS may raise a process's effective priority, or treat "waited too long" as a promotion. **Ageing cures starvation only**: it has no effect on a deadlock, where nothing is progressing and no priority is ever re-evaluated. A pure priority queue without ageing is rejected in nearly every real design.

9 Preemptive: SRTF & Round Robin

Definition: **SRTF** preempts the running process whenever a newly arrived job has a shorter remaining time. **Round Robin** keeps a circular **ready queue** and preempts after every **time quantum q**; a process with time left goes to the **back** of the queue.



round robin with q = 4 ms; all three arrived at 0, bursts 24 / 3 / 3 — waiting time = completion − arrival − burst

Worked example 1 ✍️ round robin, q = 4: P1 = 24, P2 = 3, P3 = 3, all arriving at 0. Gantt: P1 0–4, P2 4–7, P3 7–10, then P1 alone 10–30. Waiting = completion − burst: P1 = 30 − 24 = 6, P2 = 7 − 3 = 4, P3 = 10 − 3 = 7 → average 17/3 = **5.67 ms**. On this set SJF would give 0, 3, 6 → **3 ms**; FCFS 17 ms.

Worked example 2 ✍️ SRTF, staggered arrivals:
P1 (0, 8), P2 (1, 4), P3 (2, 9), P4 (3, 5). At t = 1, P2 (4) beats P1's remaining 7 → preempt; at t = 5 P4 (5) beats P1 (7) and P3 (9); at t = 10 P1 (7) beats P3 (9). Gantt: **P1 0–1 | P2 1–5 | P4 5–10 | P1 10–17 | P3 17–26**. Completion 17, 5, 26, 10 → turnaround 17, 4, 24, 7 → average **13 ms**; waiting 9, 0, 15, 2 → average **6.5 ms**.

Time quantum q	Behaviour
q → large (q ≥ burst)	no preemption in practice → **RR degenerates into FCFS**
q → small	many context switches; most time is spent switching, throughput falls
q just above the average burst	the usual choice — fast enough to be interactive, slow enough to batch most work in a few turns
Worst-case response	bounded by **(n − 1) × q** for n ready processes

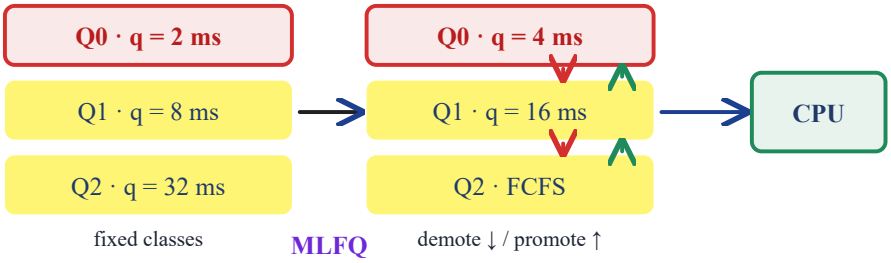
Memory trick: "The quantum is a **promise about waiting**: small q promises fairness and a quick first response and pays in switches; large q promises efficiency and pays in response time."

Exam trap: (i) In RR, waiting time is **completion − arrival − burst**, not "time in the queue" counted per quantum. (ii) A process that finishes inside its quantum **never waits again**. (iii) RR is **preemptive**; "run to completion in slices" describes FCFS.

Thread vs process scheduling, once more: If each process holds one thread, RR gives each **process** a turn — unfair when one process owns fifty threads. Modern kernels therefore schedule **threads**, so every runnable thread gets its own slice.

10 Multilevel Queues, Feedback & Real-Time

Definition: A **multilevel queue** keeps permanent classes — system processes with a short quantum and round robin, user processes with FCFS or SJF. A **multilevel feedback queue (MLFQ)** allows movement: a process that burns its whole quantum is demoted, one that waits long enough is promoted.



the whole difference: may a process **change queue**?

Point	Multilevel queue	Multilevel feedback queue
Movement	**none** — the class is fixed	a process is promoted or demoted
Quantum per queue	one per class	grows as priority falls
Strength	simple and cheap; a different admission policy per class	separates **interactive** from **CPU-bound** automatically
Weakness	a low class can **starve**	needs promotion rules to avoid starvation

Real-time policy	Priority rule
Rate monotonic	**static**: shorter period → higher priority
EDF	**dynamic**: earliest deadline first

Memory trick: "**Fixed classes don't move; feedback queues teach the scheduler.**" And "**rate** is fixed, **deadline** is re-read every instant."

Hard real-time in one line: A periodic set of n tasks on **one** processor needs $C \le T$ for every task **and** total utilisation $\le$ **$n(2^{1/n} - 1)$**: 1 task 100 %, 2 tasks 83 %, 3 tasks 78 %.

Exam trap: **EDF is optimal** — it misses a deadline only when no feasible schedule exists. Rate monotonic is optimal only for **static** priorities and loses to EDF above about 76 % utilisation.

Exam trap: The long scheduler services only the **highest** queue when it is non-empty, so low classes can wait for ever — a starvation failure that **ageing** or promotion fixes.

Worked example ✍️ two periodic tasks: A control loop needs 10 ms every 40 ms; a logger needs 5 ms every 100 ms. **(a)** Utilisation = 10/40 + 5/100 = 0.25 + 0.05 = **0.30**. **(b)** Two tasks, so the rate-monotonic bound is $2(2^{1/2} - 1) \approx$ **0.828**; 0.30 < 0.828, so the set is **feasible under rate monotonic**. **(c)** EDF meets the same deadlines and could absorb far more load. **(d)** If the loop's period shrank to 12 ms, utilisation ≈ 0.883 > 0.828, so rate monotonic can no longer guarantee it and EDF must be used.

Night-Before-the-Exam Sheet

Fact sheet:

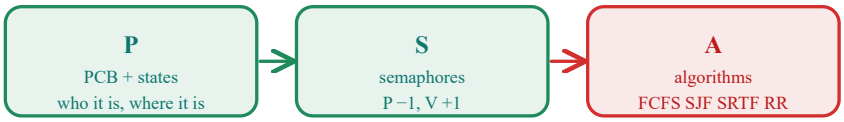
program = passive file · process = executing	process = code, data, PC, registers
PID identifies; reused after release	PCB: state, PC, regs, sched, mem, acc
new → ready → running → waiting → terminated	ready = wants the CPU, waiting = an event
long-term = job queue · medium-term = swap	short-term = the CPU scheduler
threads share the address space, own stacks	user-level blocks the process
many-to-one · one-to-one · many-to-many	kernel-level blocks one thread
IPC: shm, pipe, FIFO, queue, socket, signal	an anonymous pipe = related processes
semaphore P = −1 (may block), V = +1	a mutex has an owner, a semaphore not
empty + full = buffer size N	producer waits empty, consumer waits full
TAT = completion − arrival	WT = TAT − burst · response = first run − arrival
SJF = minimum average waiting time	FCFS = convoy · priority = starvation
RR: q large → FCFS, q small → switches	RR worst-case response = (n − 1)q
MLFQ demotes CPU-bound, promotes the idle	EDF optimal · RM bound $n(2^{1/n} - 1)$

Top 12 one-liners for MCQs

1. Five states, not four — **new** is separate
2. Ready = waiting **only** for the CPU
3. Three schedulers: long, medium, short
4. The PCB is a **kernel** structure
5. Threads share the address space, not the stack
6. Many-to-one: a blocking call stops **all** threads
7. Shared memory is fastest but needs **your own** lock
8. P decrements, V increments
9. SJF is optimal for **average waiting**
10. FCFS's disease is the **convoy**
11. RR degenerates into FCFS as q grows
12. EDF optimal; RM bound $n(2^{1/n} - 1)$

Top traps: ready vs waiting · WT ≠ TAT, response ≠ WT · SJF optimal for average **waiting** only · a new process enters the **ready** queue, not running · anonymous pipes cannot cross unrelated processes · shared memory carries **no** mutual exclusion · unlocking a mutex you do not own is meaningless · ageing cures starvation, never deadlock · FCFS does not starve, priority and SJF do · the real-time bound for two tasks is 83 %, not 100 %.

All tricks together: "Program is a recipe, process is the dish plus the ticket" · "Ready asks for the CPU, waiting asks for something else" · "Born ready, die finished" · "Threads share, they do not copy" · "Copy = message passing, map = shared memory, named = unrelated" · "the semaphore pays out on P" · "empty + full always = N" · "short first, long last" · "the quantum is a promise about waiting" · "fixed classes don't move; feedback queues teach the scheduler" · "rate is fixed, deadline is re-read every instant".



every scheduling question is a Gantt chart plus a subtraction

process, synchronisation, scheduling — the three halves of this lesson

✓ Practice MCQs (exam pattern, 10 marks)

1. In memory, waiting only for the CPU:
(a) New (b) Waiting
(c) Terminated (d) Ready

2. A newly created process goes to:
(a) running (b) a device queue
(c) waiting (d) ready queue

3. Which is **not** a PCB field?
(a) program counter (b) CPU registers
(c) last disk sector (d) process state

4. A blocked process needing only the CPU is:
(a) preempted (b) terminated
(c) dispatched (d) admitted

5. Round Robin mainly serves:
(a) batch (b) time-sharing
(c) single-tasking (d) spooling

6. Threads of one process share:
(a) stacks (b) address space
(c) registers (d) PC values

7. S = 7, then 10 P and 6 V gives:
(a) 3 (b) 1
(c) 13 (d) 0

8. Bursts 24, 3, 3 at t = 0; FCFS average wait:
(a) 3 ms (b) 13 ms
(c) 17 ms (d) 27 ms

9. Same bursts, q = 4, RR average wait:
(a) 4.00 ms (b) 5.67 ms
(c) 7.33 ms (d) 17.00 ms

10. A child whose parent died first is:
(a) a zombie (b) an orphan
(c) a daemon (d) a kernel thread

Answer key → 1-d · 2-d · 3-c · 4-c · 5-b · 6-b · 7-a · 8-c · 9-b · 10-b

Written question: What is a process, and what does the Process Control Block contain? Draw the five-state transition diagram with all seven arrows and say which queue a process in main memory waiting only for the CPU belongs to. Bursts P1 = 24, P2 = 3, P3 = 3 ms all arrive at t = 0: find the average waiting time under FCFS and under round robin with q = 4 ms, and name the algorithm optimal for average waiting time. 10

1. **Process and PCB** — a program loaded into memory and executing, with its code, data, program counter, registers and open files. PCB = state, program counter, CPU registers, CPU-scheduling information, memory-management information, accounting information. 3
2. **The diagram** — new → ready (admitted); ready → running (dispatch); running → ready (interrupt or slice ends); running → waiting (I/O or event wait); waiting → ready (completion); running → terminated; new → terminated (admission refused). Waiting only for the CPU ⇒ **ready queue**, not a device queue. 3

3 **FCFS** — P1 0–24, P2 24–27, P3 27–30. Waiting 0, 24, 27 → average 51/3 = **17 ms**. 2

4 **RR, q = 4** — P1 0–4, P2 4–7, P3 7–10, then P1 alone 10–30. Waiting = completion − burst: 6, 4, 7 → average 17/3 = **5.67 ms**. Optimal for average waiting: **SJF** (SRTF preemptively) — 0, 3, 6 → **3 ms**. 2

Before the exam ✓

- ✓ Can name the five states and all seven transitions
- ✓ Can list the PCB fields without looking
- ✓ Can draw a Gantt chart for FCFS, SJF and RR
- ✓ Can trace a semaphore through producer–consumer